Optimization Methods for Static Memory Planning in TinyML Inference

Author

Yihe Lin - 03813087

Xinrui Liu - 03797445

Course

Multi-Criteria Optimization and Decision Analysis for Embedded Systems Design

March 13, 2026

Contents

1 Introduction **1**

2 Dataset Generation and Scenario Design **1**
2.1 Neural Network Topology Motivation 1
2.2 Dataset Generation 2

3 Baseline Memory Planning Methods **3**

4 Meta-heuristic Algorithms **6**

5 Experimental Results and Analysis **8**

6 Conclusion **10**

1 Introduction

Deploying neural networks on embedded devices is a central challenge in TinyML, where hardware platforms typically provide only limited on-chip memory. During inference, each layer produces intermediate results called activation tensors, which must remain stored in memory until they are consumed by subsequent operations. In the deployment pipeline, neural network execution can be represented as a data flow graph (DFG), where nodes correspond to operators and edges correspond to activation tensors.

Because embedded devices have very limited SRAM, efficiently managing these intermediate tensors is critical. Many tensors are not simultaneously active during execution, meaning that memory regions can be reused when their lifetimes do not overlap. Static memory planning therefore aims to assign tensors to memory locations while minimizing the peak memory usage required during inference. This task can be viewed as a combinatorial optimization problem defined by tensor sizes and lifetime intervals.

In this report, we investigate optimization methods for static memory planning. First we establish several foundational storage methods, including Ping-pong allocation, Greedy-by-size buffer sharing, and Hole-aware Greedy placement, in order to compare discrete-buffer and offset-based memory allocation strategies. Based on the observations obtained from these baseline methods, we then apply Local Search, Simulated Annealing, and Genetic Algorithms to further optimize the allocation process.

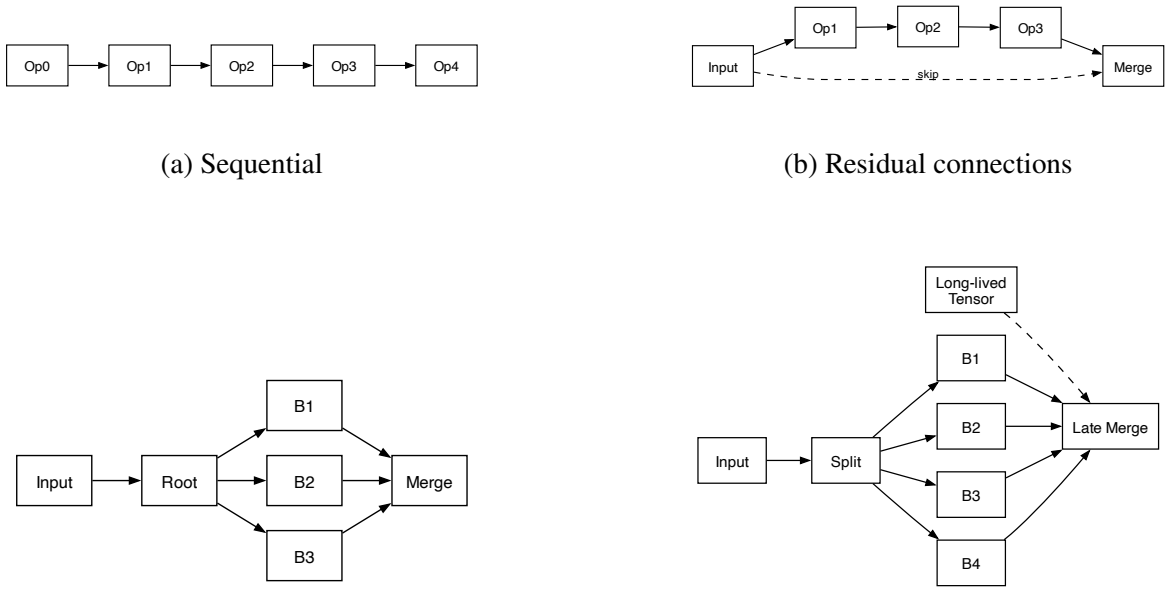
Rather than evaluating the methods only in terms of final peak memory, we analyze the trade-off between memory reduction and computational effort across different neural network execution patterns. This allows the report to identify not only which methods achieve low memory footprints, but also which strategies provide the most practical balance between optimization quality and runtime cost for TinyML deployment.

2 Dataset Generation and Scenario Design

2.1 Neural Network Topology Motivation

Different neural network architectures lead to different activation lifetime patterns, which directly affect the difficulty of static memory planning. In a simple sequential network, each layer consumes the output of the previous layer and produces a new activation tensor that is used shortly afterwards. As a result, tensor lifetimes are short and overlap only locally, making memory reuse relatively easy.

More complex neural network architectures introduce structures that increase memory pressure. Residual connections, commonly used in ResNet-style networks, reuse outputs from earlier layers through skip connections. These tensors must remain alive until a later merge operation, which reduces opportunities for memory reuse. Another common pattern is the multi-branch architecture, such as in Inception-style networks, where the computation splits into several parallel branches before merging the results. In this case, multiple activation tensors may exist in memory at



(a) Sequential

(b) Residual connections

(c) Fork–join branching

(d) Late-merge structure

Figure 1: Representative DAG patterns used in the dataset generator: (a) sequential execution, (b) residual connections with skip edges, (c) fork–join branching, and (d) late-merge architectures.

the same time, creating more lifetime overlap. In addition, some architectures merge these branches only at a later stage. In such late-merge structures, tensors remain alive for longer periods, which increases peak memory usage.

Figure 1 shows simplified DAG representations of the four topology patterns used in this work: sequential execution, residual connections, fork–join branching, and late-merge structures. These patterns correspond to increasing levels of tensor lifetime overlap and therefore increasing difficulty for static memory allocation.

These four patterns represent increasing levels of memory allocation difficulty, from simple sequential execution to multi-branch structures with long-lived tensors. Evaluating allocation algorithms on these different structures helps us understand how network topology affects memory reuse and peak memory usage.

2.2 Dataset Generation

To evaluate the memory planning algorithms, we implemented a dataset generator that produces activation tensor records following the topology patterns described above. Each tensor record contains four attributes: a tensor identifier, its aligned memory size in bytes, the operator index where the tensor becomes live, and the operator index where it is last used. These attributes are sufficient to model tensor lifetimes for static memory planning.

Four topology families are generated: *sequential*, *residual*, *fork–join*, and *late-merge*. The sequential datasets represent simple chain-like execution with short tensor lifetimes. The residual

Table 1: Summary of generated dataset topologies

Topology	Instances	Tensors (approx.)	Main Feature
Sequential	10	12–16	Short lifetimes
Residual	10	14–18	Long skip tensors
Fork–Join	10	15–20	Branch overlap
Late Merge	10	16–20	Long-lived tensors

datasets introduce several long-lived tensors that simulate skip connections. The fork–join datasets model multi-branch computation where tensors from different branches overlap in time. The late-merge datasets further increase the difficulty by allowing several tensors to remain alive until a late merge stage.

For each topology family, ten dataset instances are generated using different random seeds, resulting in a total of forty benchmark instances, as summarized in Table 1. Each instance contains approximately 12–20 tensors, depending on the topology structure. Tensor sizes are sampled from predefined ranges with small random variations and are aligned to 16-byte boundaries to reflect typical embedded memory alignment constraints.

3 Baseline Memory Planning Methods

The research began by establishing three foundational storage methods.

First, the Ping-pong algorithm, which is a fundamental double-buffering mechanism. When processing a neural network's computation graph, it strictly allocates two independent memory blocks, Buffer A and Buffer B. During inference execution, the current operator reads its input data from Buffer A and writes its computed output into Buffer B. The subsequent operator then reverses these roles, reading from Buffer B and writing to Buffer A. This alternating cycle continues throughout the linear execution. Although this mechanism extremely simplifies memory management and works efficiently for strictly sequential networks, it inherently struggles with complex topologies involving long skip connections such as residual networks and parallel branches, often resulting in severe memory fragmentation or unfeasible allocations.

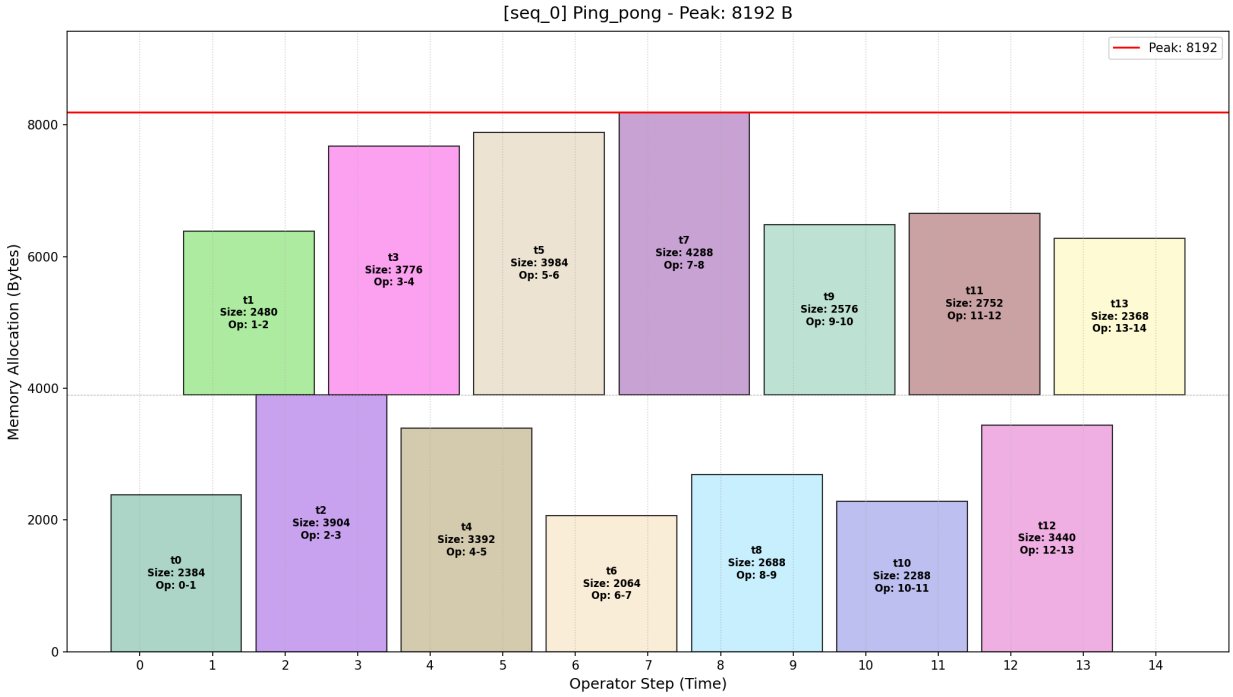


Figure 2: Memory allocation of Ping-pong algorithm in sequential networks

Secondly, the Greedy by Size algorithm. It employs a discrete bin-packing strategy for memory allocation. The algorithm begins by sorting all tensors in the network in descending order based on their memory footprint, prioritizing the largest tensors first. During the allocation phase, it maintains a dynamic list of discrete shared buffers. For each pending tensor, the algorithm iterates through the existing buffers to check if the tensor's life-cycle temporally overlaps with any tensors already residing in that buffer. If a buffer without temporal conflicts is found, the tensor is assigned to it; if all existing buffers exhibit life-cycle conflicts, an entirely new buffer is created. While this approach is computationally extremely fast, it suffers from internal fragmentation, as the capacity of each buffer is strictly dictated by its largest resident tensor, leaving unused gaps when smaller tensors occupy the same buffer.

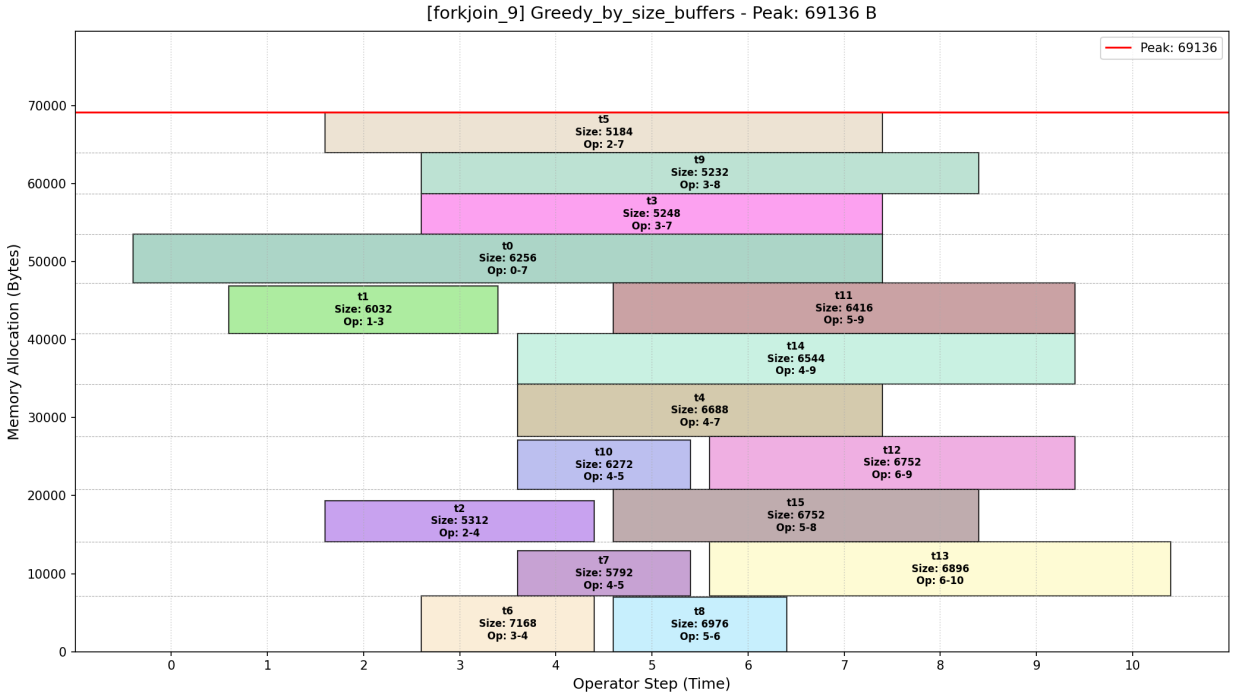


Figure 3: Memory allocation of Greedy by Size algorithm in forkjoin networks

Thirdly, the Hole-aware Greedy algorithm, which utilizes an advanced continuous memory placement strategy. Unlike the discrete buffers approach, it treats the entire physical memory as a single, contiguous address space (offset). Processing tensors in descending order of size, the algorithm dynamically identifies all previously allocated tensors whose life-cycles temporally overlap with the current tensor. Then it sorted these overlapping tensors by their physical memory offsets. Subsequently, the algorithm precisely measures the spatial gaps between adjacent tensors and employs a "Best-fit" strategy to locate the smallest continuous hole capable of accommodating the new tensor. If no such hole exists, the tensor is placed at the current peak memory boundary. By packing tensors like playing the Tetris game, this strategy aggressively minimizes memory fragmentation and achieves a significantly higher theoretical spatial utilization rate.

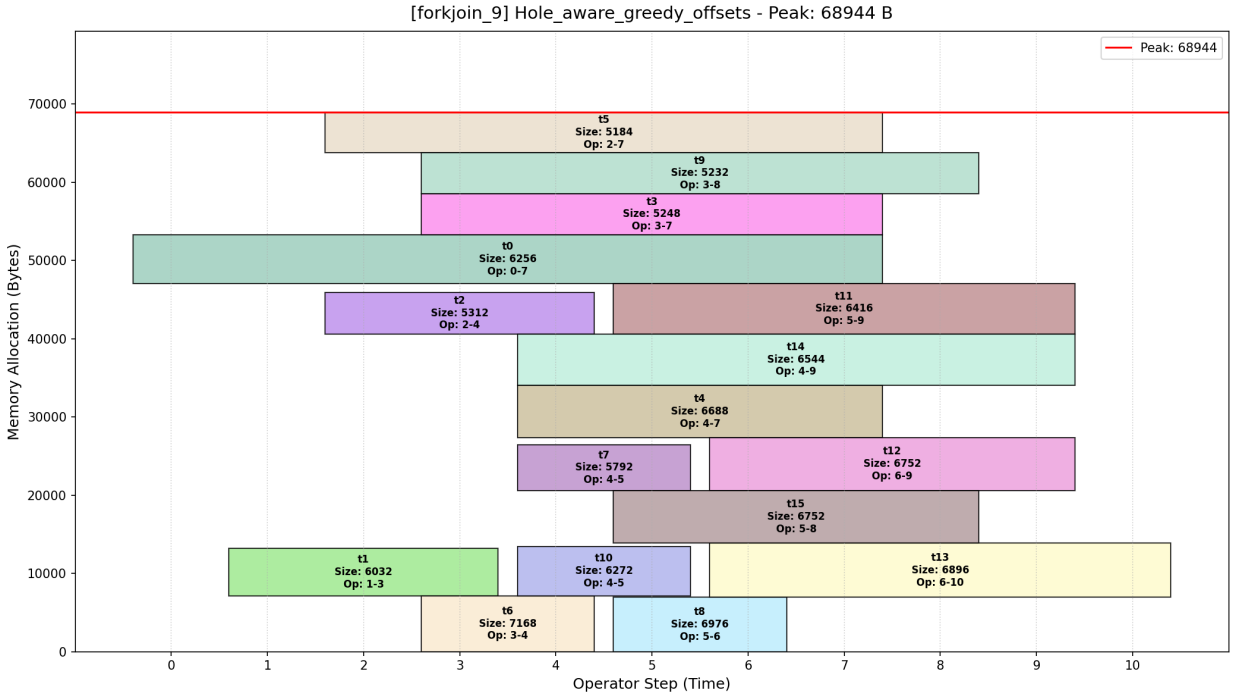


Figure 4: Memory allocation of Hole-aware Greedy algorithm in forkjoin networks

By comparing Figure 3 and Figure 4, it is evident that when processing the same task, the Hole-aware Greedy algorithm can achieve a memory allocation strategy with a lower peak memory than the Greedy by Size algorithm.

4 Meta-heuristic Algorithms

To further compress the peak memory footprint, we introduced three meta-heuristic algorithms—Local Search, Simulated Annealing (SA), and Genetic Algorithms (GA) to deeply optimize the initial solutions generated by the Greedy by Size method. During this phase, to facilitate rigorous research and analysis, we developed an automated hyperparameter tuning program and an intuitive memory allocation visualization tool. However, a critical observation emerged during the optimization process: the unoptimized Hole-aware Greedy algorithm would occasionally discover exceptionally superior solutions, outperforming even the heavily optimized Greedy by Size configurations, as shown in Table 2.

By using our visualization tool, we uncover the fundamental reason behind this discrepancy: the two methods rely on entirely different underlying data structures. While Greedy by Size operates on discrete buffers, Hole-aware operates on a contiguous memory space (offsets). Even in instances where both methods incidentally achieved the exact same peak memory, as shown in Table 2, their internal spatial storage layouts were entirely distinct, with the continuous offset mechanism demonstrating a significantly higher theoretical capacity for dense packing.

Table 2: Peak memory comparison on a fork–join topology instance

Method	Allocation Model	Peak Memory (Bytes)
Hole-aware Greedy	Offsets	68944
Greedy-by-size	Buffers	69136
Local Search	Buffers	69136
Simulated Annealing	Buffers	69136
Genetic Algorithm	Buffers	69136

Capitalizing on this pivotal insight, we shifted our paradigm and used Local Search, SA, and GA directly on the Hole-aware Greedy algorithm. By allowing these advanced algorithms to search for the optimal "tensor feed sequence" rather than intervening in physical placement constraints, and coupling them with the smart hole-aware decoder, we successfully pushed past previous limits and achieved substantially lower peak memory bounds. Finally, to provide a clear, intuitive, and comprehensive evaluation of the peak memory versus time consumption trade-offs across all eight algorithmic variants, we developed a plotting script to generate Pareto charts. The resulting Pareto front systematically visualizes the performance boundaries of each algorithm, offering a robust and practical guide for selecting the optimal static memory planning strategy under varying time and space constraints in real-world compiler scenarios.

5 Experimental Results and Analysis

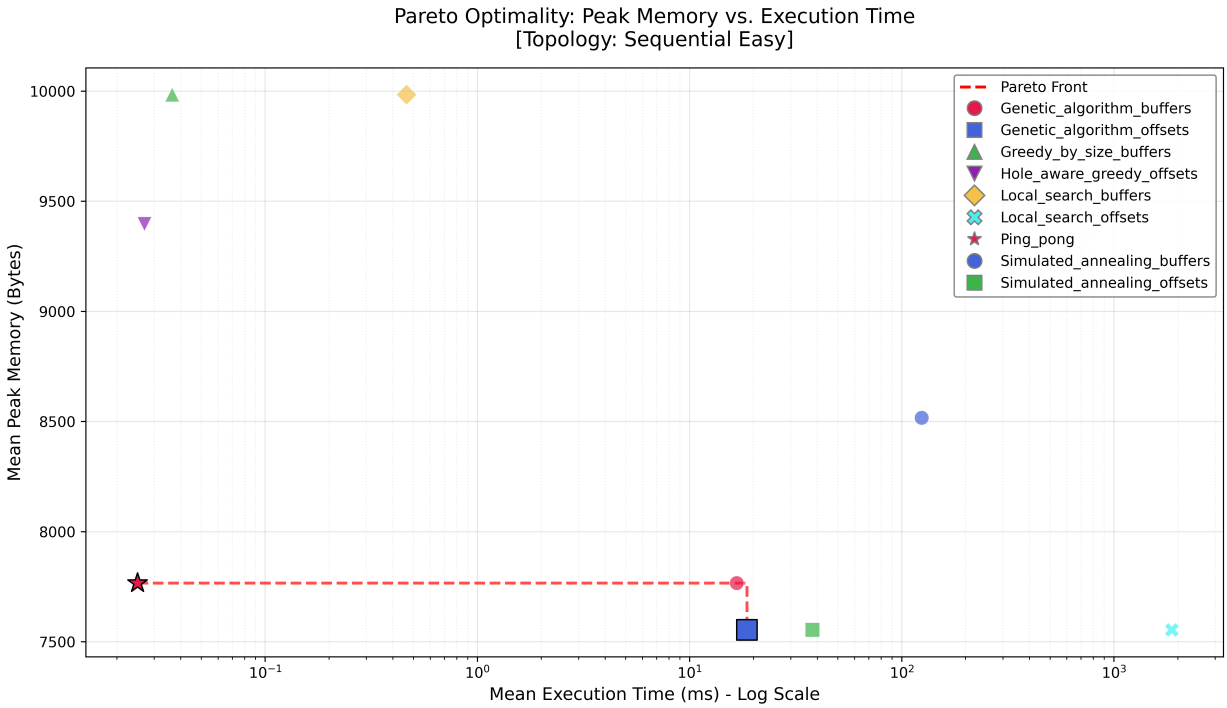


Figure 5: Memory allocation of Hole-aware Greedy algorithm in forkjoin networks

Figure 5 illustrates the Pareto front with respect to mean execution time and mean maximum memory consumption under 10 different Sequential topologies. The Ping-pong algorithm emerges as the most efficient way, occupying the ideal bottom-left region with sub-millisecond latency (0.025 ms) and a competitive memory footprint. Furthermore, Genetic Algorithm based on offsets achieves the minimum peak memory consumption across all tested candidates. However, as a meta-heuristic approach, it necessitates a longer runtime to converge compared to the more direct Ping-pong strategy.

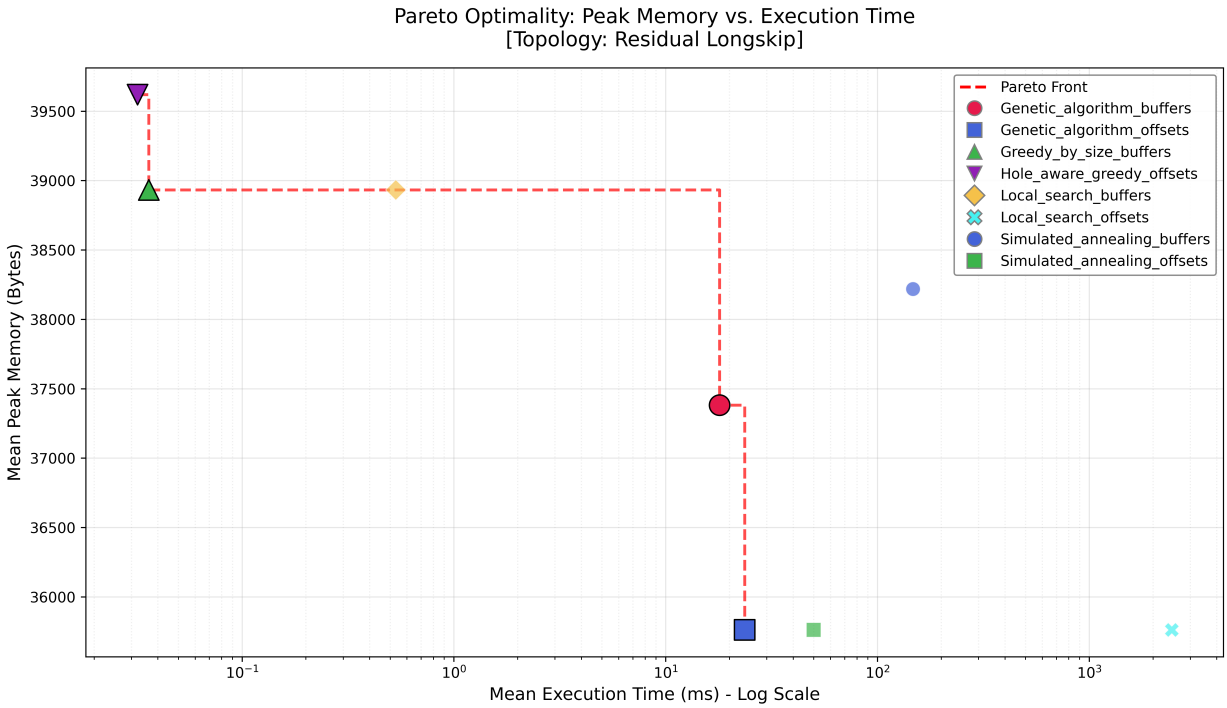


Figure 6: Memory allocation of Hole-aware Greedy algorithm in forkjoin networks

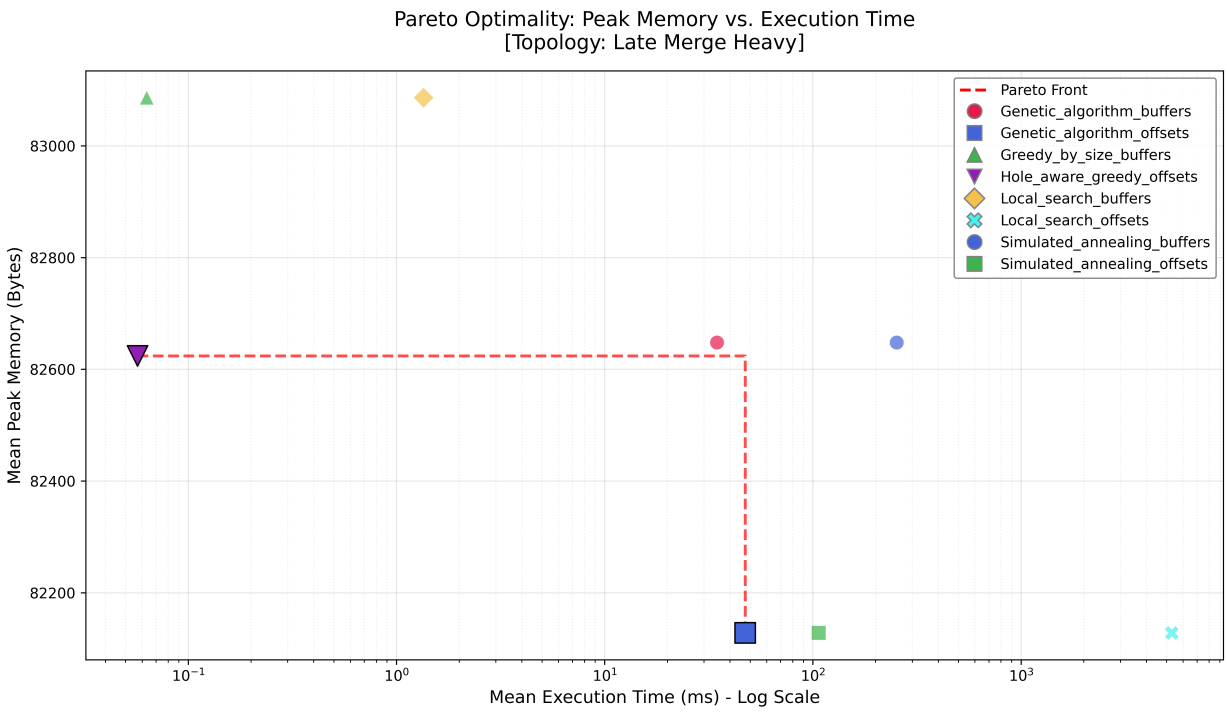


Figure 7: Memory allocation of Hole-aware Greedy algorithm in forkjoin networks

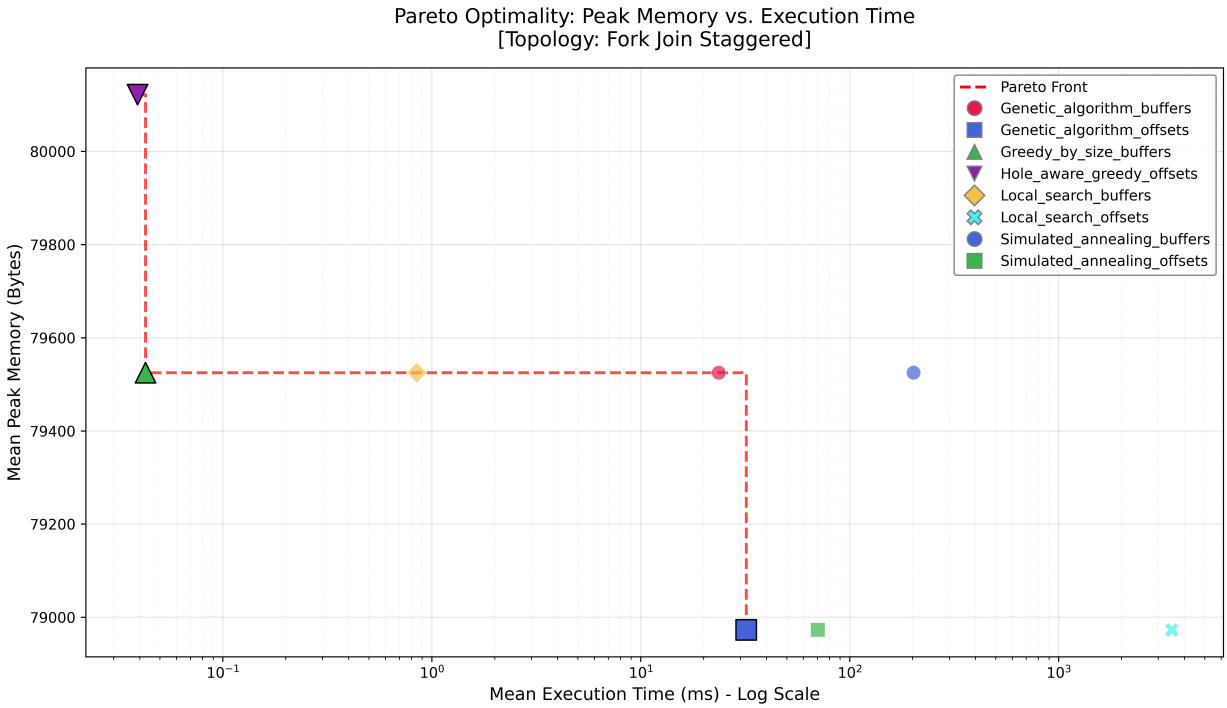


Figure 8: Memory allocation of Hole-aware Greedy algorithm in forkjoin networks

In other three examined topologies-Fork Join, Late Merge, and Residual networks-a consistent trend is observed. The Hole-aware Greedy algorithm offers the fastest execution speed. In most cases, it successfully identifies memory layouts with a lower peak memory than standard Greedy by size algorithms, although its superiority is not absolute across all complex topologies.

Among the evaluated meta-heuristic algorithms, the Genetic Algorithm demonstrates superior performance, achieving the most effective optimization with the highest computational efficiency. In contrast, Local Search and Simulated Annealing exhibit a tendency to become trapped in local optima; even in scenarios where they eventually reach a solution comparable to that of the GA, they necessitate significantly longer convergence times.

Overall, the results confirm that while the structural complexity of the computation graph dictates the baseline memory requirement, the GA-based meta-heuristic combined with an offset-based allocation represents the most robust approach for reaching the theoretical limit of resource efficiency. In the Fork Join Staggered topology, Greedy by size emerges as a viable candidate for balancing peak memory and computational overhead. Similarly, in Residual structures, both Greedy by size and Genetic Algorithm based on buffers provide competitive trade-offs.

6 Conclusion

This report studied optimization strategies for static memory planning in TinyML inference, where minimizing the peak memory footprint of activation tensors is critical for deployment on resource-constrained embedded devices.

The experimental results show that the choice of memory allocation model significantly influences the achievable memory efficiency. Offset-based allocation methods, such as the Hole-aware Greedy algorithm, consistently achieve lower peak memory consumption than buffer-based approaches because tensors can be packed more densely in a continuous memory space.

Meta-heuristic optimization further improves the allocation quality by exploring different tensor ordering strategies. Among the evaluated approaches, the Genetic Algorithm provides the most effective memory reduction across most network topologies, although it requires longer runtime compared with simple greedy methods. In contrast, the Ping-pong strategy achieves the fastest execution time and remains competitive for simple sequential networks.

Overall, the results indicate that static memory planning performance depends both on the allocation model and the optimization strategy. Combining offset-based allocation with meta-heuristic search provides an effective approach for reducing peak memory usage in TinyML deployment scenarios.